

Inverse Protein Folding Challenge — Reconstructing Sequence from Structure

Background

Proteins are the primary executors of life activities, consisting of long-chain molecules made up of 20 types of amino acids in a specific order. Under natural conditions, they spontaneously fold into specific three-dimensional structures. Understanding how amino acid sequences determine the three-dimensional structure of proteins—the "protein folding problem"—has remained a scientific challenge for over half a century. In recent years, deep learning models represented by AlphaFold have achieved breakthroughs in this area: given an amino acid sequence, the model can predict its folded three-dimensional structure with high accuracy.

Yet nature presents a puzzle in the opposite direction: given a three-dimensional structure, can you deduce its original amino acid sequence? This problem is known as **"inverse protein folding"** and is a core challenge in protein engineering and drug design. Imagine a scientist designing a protein scaffold that could theoretically bind perfectly to a disease-causing target, but not knowing which amino acids should be used to build it—inverse folding algorithms are the "decoding" key that helps identify the amino acid sequences capable of folding into the target structure. Its practical significance is profound: in targeted cancer therapy, inverse folding techniques can be used to design high-affinity antibody drugs; in synthetic biology, it facilitates the creation of novel enzymes not found in nature for biofuel production or plastic degradation; and in vaccine development, it enables scientists to design more stable immunogens based on viral surface structures.

The task in this problem is: given the spatial coordinates of each residue (amino acid unit) in a protein's three-dimensional structure, predict which amino acid should be placed at each position. **The evaluation metric is solely the F_2 score (F2 Score)**, focusing on prediction performance for rare amino acid classes—because certain amino acids occur less frequently in natural proteins but often play critical roles in drug design.

Problem Description

A protein is composed of a sequence of amino acids. Each amino acid residue contains an α -carbon atom (C_α). In three-dimensional space, the backbone of the protein can be described by the 3D coordinates (x, y, z) of these C_α atoms. The goal of the inverse folding problem is exactly: based on

the positional relationships of these C_α atoms, predict which amino acid should be placed at each position.

In this problem, you need to accomplish the following tasks:

1. Build a model;
2. Input: sequence of C_α atom coordinates of the protein;
3. Output: amino acid class at each position (20 standard amino acids);
4. Model training and prediction;
5. **Final ranking is based solely on the F_2 score of the model's predictions on the B-set dataset compared to the ground truth.**

The length of each protein (number of residues) is L , and the coordinate data has the shape $L \times 3$. Amino acid classes are encoded as integers from 0 to 19, correspond to the 20 standard amino acids.

Dataset

The data for this problem comes from the CATH 4.4 non-redundant dataset (sequence identity $\leq 40\%$), preprocessed into three CSV files. Protein lengths L range from 50 to 800, with an average length of approximately 200. **Each sample has had residues with missing C_α coordinates removed**, so the coordinate sequence and amino acid label sequence in the CSV files have exactly the same length, and every residue has complete 3D coordinates.

File Descriptions

- **train.csv** : Training set, containing 30000 samples, providing complete coordinates and amino acid labels.
- **A.csv** : A test set, containing 2000 samples, providing only coordinates; labels are not public, and scores are not disclosed. Used for model selection.
- **B.csv** : B test set, containing 1938 samples, providing only coordinates; labels are not public, and scores will be disclosed after the competition ends. Used for final scoring.

CSV Format

Each CSV file uses comma separation, UTF-8 encoding, with a header row as column names.

Columns of train.csv :

Column Name	Data Type	Description
domain_id	string	Unique identifier of the protein domain (e.g., 12asA00 , case-sensitive)
coords	string	3D coordinates of all C_{α} atoms, flattened in order as a comma-separated string: $x_1, y_1, z_1, x_2, y_2, z_2, \dots, x_L, y_L, z_L$, each coordinate with 3 decimal places
labels	string	Amino acid class encoding for each residue (0–19), comma-separated in order, e.g., 5,12,3,19,1,...

Columns of A.csv and B.csv :

Column Name	Data Type	Description
domain_id	string	Unique identifier of the protein domain (e.g., 12asA00 , case-sensitive)
coords	string	3D coordinates of all C_{α} atoms, flattened in order as a comma-separated string: $x_1, y_1, z_1, x_2, y_2, z_2, \dots, x_L, y_L, z_L$, each coordinate with 3 decimal places

Note: The test set files do not contain the `labels` column. Participants need to predict the amino acid class for each position of each sample themselves and output the results according to the submission format requirements.

Amino Acid Class Encoding Table

Code	Amino Acid	Three-letter	One-letter
0	Alanine	Ala	A
1	Cysteine	Cys	C
2	Aspartic acid	Asp	D
3	Glutamic acid	Glu	E
4	Phenylalanine	Phe	F
5	Glycine	Gly	G
6	Histidine	His	H

Code	Amino Acid	Three-letter	One-letter
7	Isoleucine	Ile	I
8	Lysine	Lys	K
9	Leucine	Leu	L
10	Methionine	Met	M
11	Asparagine	Asn	N
12	Proline	Pro	P
13	Glutamine	Gln	Q
14	Arginine	Arg	R
15	Serine	Ser	S
16	Threonine	Thr	T
17	Valine	Val	V
18	Tryptophan	Trp	W
19	Tyrosine	Tyr	Y

Conversion table that can be used directly in code:

```
aa2id = {
    'A': 0, 'C': 1, 'D': 2, 'E': 3, 'F': 4, 'G': 5, 'H': 6, 'I': 7, 'K': 8, 'L': 9,
    'M': 10, 'N': 11, 'P': 12, 'Q': 13, 'R': 14, 'S': 15, 'T': 16, 'V': 17, 'W': 18, 'Y': 19
}
id2aa = {
    0: 'A', 1: 'C', 2: 'D', 3: 'E', 4: 'F', 5: 'G', 6: 'H', 7: 'I', 8: 'K', 9: 'L',
    10: 'M', 11: 'N', 12: 'P', 13: 'Q', 14: 'R', 15: 'S', 16: 'T', 17: 'V', 18: 'W', 19: 'Y'
}
```

Evaluation Metric

This problem uses the F_2 **score (F2 Score)** as the sole evaluation metric. The F_2 score is the weighted harmonic mean of precision and recall, where recall is weighted 4 times as much as precision, defined as:

$$F_2 = (1 + 2^2) \cdot \frac{\text{Precision} \cdot \text{Recall}}{(2^2 \cdot \text{Precision}) + \text{Recall}} = 5 \cdot \frac{\text{Precision} \cdot \text{Recall}}{4 \cdot \text{Precision} + \text{Recall}}$$

The F_2 score places greater emphasis on recall, meaning it expects the model to identify as many correct amino acids as possible (especially those rare but important classes), penalizing missed detections more severely than false alarms. This metric reflects the practical requirements of protein inverse folding: **missing a critical functional site could render the entire protein ineffective; occasionally mispredicting a non-critical position might still allow the protein to function.**

Calculation Method: For each protein sample in the test set, calculate the precision and recall for each amino acid class separately, then compute the F_2 score for that class; finally, take the **macro average** across all classes to obtain the F_2 score for that sample. The F_2 scores of all samples are then arithmetically averaged to produce the final evaluation metric. If a particular class does not appear in a sample, that class is skipped (i.e., it does not participate in the macro average for that sample).

Submission Format

Participants need to generate corresponding prediction result files for `A.csv` and `B.csv`, named `A_predict.csv` and `B_predict.csv`, respectively. Each prediction file must contain the following two columns:

Column Name	Data Type	Description
<code>domain_id</code>	string	Exactly matches the <code>domain_id</code> from the original test file
<code>predictions</code>	string	Predicted amino acid class for each residue (0 ~ 19), comma-separated in order. The length must equal the number of residues (i.e., number of coordinate triplets) in the original <code>coords</code> column

Note: The length of the prediction result must strictly equal the number of residues L for that protein; otherwise, that entry will receive no credit.

Constraints

1. The use of any pre-trained model weights is prohibited (including but not limited to image pre-trained models like EfficientNet, ResNet, as well as protein language models such as ESM, ProtBERT, AlphaFold). Participants must train their own models from random initialization. However, the **framework code** (i.e., network architecture) of models like EfficientNet or ResNet may be used.

2. Calling third-party libraries not officially provided by the NOAI platform is prohibited, including but not limited to `mmproteo` and `capipy`.
3. Calling external LLM APIs (e.g., GPT, Claude) for prediction or feature generation assistance is prohibited.
4. Please design model parameters reasonably (recommended within 10^6) and training time (recommended not exceeding 10 minutes).
5. Submission contents: two files, `A_predict.csv` and `B_predict.csv`.

References

- CATH 4.4 database paper: Waman, V. P., et al. (2025). CATH v4.4: major expansion of CATH by experimental and predicted structural data. *Nucleic Acids Research*, 53(D1), D348–D355. doi:[10.1093/nar/gkae1087](https://doi.org/10.1093/nar/gkae1087), included in the dataset archive.
- Inverse folding related methods: PiFold, ProteinMPNN, etc. (ArXiv preprints can be consulted independently).

Data Source and License

The dataset used in this problem is derived from the CATH 4.4 non-redundant dataset (Waman et al., 2025) through filtering and processing. The original CATH 4.4 data is released by the [UCL research team](#) under the [CC BY 4.0](#) license.

We have applied the following processing: residues with missing $\text{C}\alpha$ coordinates have been removed, and the sequential coordinates together with amino acid labels have been organized into CSV format on a per-sample basis.

The problem statement, its design, and the derived portions of the dataset were created by [cyx012113](#) and are licensed as a whole under [CC BY-NC-SA 4.0](#).

This means you are free to share and adapt this problem, provided that you give appropriate credit, do not use it for commercial purposes, and any adaptations must be distributed under the same license.

- Original CATH 4.4 license: <https://creativecommons.org/licenses/by/4.0>
- This problem's license: <https://creativecommons.org/licenses/by-nc-sa/4.0>